



MKD

2026-08-03

- 1 Contest
- 2 Mathematics
- 3 Data structures
- 4 Numerical
- 5 Number theory
- 6 Graph
- 7 Geometry
- 8 Strings
- 9 Various

Contest (1)

dbg.h20 lines

```
#define dbg(...) cerr<<"DEBUG:["<<#__VA_ARGS__<<"] = [" ,\n\
_print(__VA_ARGS__),cerr<<"]\n"
template<class T, class=void>struct cntr : false_type {};
template<class T>struct cntr<T,
void_t<decltype(declval<T>().begin()),
decltype(declval<T>().end()),
typename T::value_type> > : true_type {};
template<class T>using raw=remove_cv_t<remove_reference_t<T> >;
template<class T>enable_if_t<!cntr<T>::value>
_print(const T &x) { cerr << x; }
template<class T>enable_if_t<cntr<T>::value>_print(const T &v){
cerr << "["; bool f = false;
for (const auto &x: v)
f ? cerr << (cntr<raw<decltype(x)> >::value ?",\n":" , "):
cerr, f = true, _print(x);
cerr << "]" ;
}
template<class T, class ... A>
void _print(const T &x, const A &... a) {
_print(x);(cerr << " , " , _print(a)), ... );}
```

template.cpp26 lines

```
#include <bits/stdc++.h>
using namespace std;
// #define LOCAL_DEBUG
#ifdef LOCAL_DEBUG
#include "dbg.h"
#else
#define dbg(...) 42
#endif
#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
using ll = long long;
using pii = pair<int, int>;
using vi = vector<int>;
ll popcount(ll x) { return __builtin_popcountll(x); }
void init() {}
void solve() {}
```

signed main() {19 lines

```
signed main() {
init();
cin.tie(nullptr), ios::sync_with_stdio(false);
// cin.exceptions(cin.failbit);
int T = 1;
// cin >> T;
while (T-->solve();
return 0;
}
```

1.1 checker

cek.sh19 lines

```
#!/usr/bin/env bash
set -u
cd "$(dirname "$0")"
g++ gen.cpp -o gen -std=c++17 -O2
g++ std.cpp -o std -std=c++17 -O2
g++ sol.cpp -o sol -std=c++17 -O2
cnt=0
while true; do
cnt=$((cnt + 1))
echo "Testing case #$cnt"
./gen "${1-}" > in.txt
./std < in.txt > out1.txt
./sol < in.txt > out2.txt
if ! diff -q out1.txt out2.txt > /dev/null; then
echo "WA on case #$cnt"
diff out1.txt out2.txt
read -r -p "Press Enter to continue..."
fi
done
```

cek.bat19 lines

```
@echo off
cd "%~dp0"
g++ gen.cpp -o gen -std=c++17 -O2
g++ std.cpp -o std -std=c++17 -O2
g++ sol.cpp -o sol -std=c++17 -O2
set cnt=0
:loop
set /a cnt+=1
echo Testing case %cnt%
gen %1 > in.txt
std < in.txt > out1.txt
sol < in.txt > out2.txt
fc out1.txt out2.txt > nul
if errorlevel 1 (
echo WA on case %cnt%
fc out1.txt out2.txt
pause
)
goto loop
```

gen.h22 lines

```
uint64_t seed = chrono::steady_clock::now()
.time_since_epoch().count() ^ random_device{} ( );
mt19937 rng(seed);
template<class T>
T randi(T l,T r){return uniform_int_distribution<T>(l,r)(rng);}
vi randvi(int n, int l, int r) {
vi v(n);
rep(i, 0, sz(v))v[i] = randi(l, r);
return v;
}
vi randp(int n) {
vi p(n);
```

iota(all(p), 1); shuffle(all(p), rng); return p; } int main(int argc, char \*argv[]) { if (argc >= 2) seed = stoull(argv[1]), rng.seed(seed); cerr << seed << endl; return 0; }

troubleshootzh.txt49 lines

**Pre-submit:**  
样例不够时写简单测试。  
时间限制紧时写大数据测试。  
内存是否足够。  
是否可能溢出。  
是否提交了正确文件。  
算法复杂度是否正确。是否有与n无关的复杂度。  
调试的改动是否改回(如len = 2e4 -> len = 20)  
变量在同一个语句内修改与调用引发UB  
(如seg[pos] = Info::merge(val, seg[pos += n]);))

**Wrong answer:**  
打印解法和调试输出。  
多组测试之间是否清空数据结构。  
算法是否覆盖完整输入范围。  
x=y, x!=y。  
是否读懂题意和所有边界情况。  
是否有未初始化变量、整数溢出。  
是否混淆 N/M 、i/j 等变量。  
确定算法是否正确，是否处理所有特殊情况。  
STL函数行为是否理解正确。  
添加断言、自造测试、手算简单样例。  
找队友解释算法或让队友看代码。  
散步、上厕所。  
检查输出格式，包括大小写、单词是否每个字母都一致、空格和换行。  
必要时重写代码。

**Runtime error:**  
是否本地测试过边界情况。  
未初始化变量。  
越界。  
断言失败。  
除以0或模0。  
无限递归。  
指针/迭代器失效。  
内存使用过大。  
通过重新提交进行调试(重映射，见Vairous)。

**Time limit exceeded:**  
是否有死循环。  
算法复杂度是否正确。  
是否有大量不必要拷贝。  
输入输出是否太大，是否需要自写快读。  
避免vector，map。改用array，unordered\_map。  
和队友确认算法是否合理。

**Memory limit exceeded:**  
算法理论最大内存。  
多组测试之间是否清空数据结构。

## Mathematics (2)

### 2.1 方程

$$ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

极值点  $x = -\frac{b}{2a}$ .

$$\begin{aligned} ax + by &= e & x &= \frac{ed - bf}{ad - bc} \\ cx + dy &= f & y &= \frac{af - ec}{ad - bc} \end{aligned}$$

一般地, 给定方程  $Ax = b$ , 变量  $x_i$  的解为

$$x_i = \frac{\det A'_i}{\det A}$$

其中  $A'_i$  表示将  $A$  的第  $i$  列替换为  $b$ .

### 2.2 三角函数

$$\begin{aligned} \sin(v + w) &= \sin v \cos w + \cos v \sin w \\ \cos(v + w) &= \cos v \cos w - \sin v \sin w \end{aligned}$$

$$\begin{aligned} \tan(v + w) &= \frac{\tan v + \tan w}{1 - \tan v \tan w} \\ \sin v + \sin w &= 2 \sin \frac{v + w}{2} \cos \frac{v - w}{2} \\ \cos v + \cos w &= 2 \cos \frac{v + w}{2} \cos \frac{v - w}{2} \end{aligned}$$

$$(V + W) \tan(v - w)/2 = (V - W) \tan(v + w)/2$$

其中  $V, W$  是角  $v, w$  所对边的长度.

$$\begin{aligned} a \cos x + b \sin x &= r \cos(x - \phi) \\ a \sin x + b \cos x &= r \sin(x + \phi) \end{aligned}$$

其中  $r = \sqrt{a^2 + b^2}$ ,  $\phi = \text{atan2}(b, a)$ .

### 2.3 几何

#### 2.3.1 三角形

边长:  $a, b, c$

$$\text{半周长: } p = \frac{a + b + c}{2}$$

$$\text{面积: } A = \sqrt{p(p-a)(p-b)(p-c)}$$

$$\text{外接圆半径: } R = \frac{abc}{4A}$$

$$\text{内切圆半径: } r = \frac{A}{p}$$

中线长度 (将三角形分为两个面积相等的三角形):

$$m_a = \frac{1}{2} \sqrt{2b^2 + 2c^2 - a^2}$$

角平分线长度 (将角度分为两个相等的角度):

$$s_a = \sqrt{bc \left[ 1 - \left( \frac{a}{b+c} \right)^2 \right]}$$

## dbg template cek cek gen troubleshootzh

$$\text{正弦定理: } \frac{\sin \alpha}{a} = \frac{\sin \beta}{b} = \frac{\sin \gamma}{c} = \frac{1}{2R}$$

$$\text{余弦定理: } a^2 = b^2 + c^2 - 2bc \cos \alpha$$

$$\text{正切定理: } \frac{a+b}{a-b} = \frac{\tan \frac{\alpha+\beta}{2}}{\tan \frac{\alpha-\beta}{2}}$$

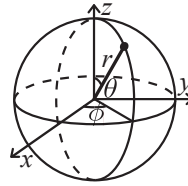
#### 2.3.2 四边形

设边长为  $a, b, c, d$ , 半周长  $p$ , 对角线为  $e, f$ , 对角线夹角  $\theta$ , 面积  $A$  和辅助量  $F = b^2 + d^2 - a^2 - c^2$ :

$$4A = 2ef \cdot \sin \theta = F \tan \theta = \sqrt{4e^2 f^2 - F^2}$$

对于圆内接四边形, 对角之和为  $180^\circ$ ,  $ef = ac + bd$ , 且  $A = \sqrt{(p-a)(p-b)(p-c)(p-d)}$ .

#### 2.3.3 球坐标



$$\begin{aligned} x &= r \sin \theta \cos \phi & r &= \sqrt{x^2 + y^2 + z^2} \\ y &= r \sin \theta \sin \phi & \theta &= \arccos(z / \sqrt{x^2 + y^2 + z^2}) \\ z &= r \cos \theta & \phi &= \text{atan2}(y, x) \end{aligned}$$

### 2.4 导数/积分

$$\begin{aligned} \frac{d}{dx} \arcsin x &= \frac{1}{\sqrt{1-x^2}} & \frac{d}{dx} \arccos x &= -\frac{1}{\sqrt{1-x^2}} \\ \frac{d}{dx} \tan x &= 1 + \tan^2 x & \frac{d}{dx} \arctan x &= \frac{1}{1+x^2} \\ \int \tan ax &= -\frac{\ln |\cos ax|}{a} & \int x \sin ax &= \frac{\sin ax - ax \cos ax}{a^2} \\ \int e^{-x^2} &= \frac{\sqrt{\pi}}{2} \text{erf}(x) & \int x e^{ax} dx &= \frac{e^{ax}}{a^2} (ax - 1) \end{aligned}$$

分部积分:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

### 2.5 求和

$$c^a + c^{a+1} + \dots + c^b = \frac{c^{b+1} - c^a}{c - 1}, c \neq 1$$

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

$$1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n(2n+1)(n+1)}{6}$$

$$1^3 + 2^3 + 3^3 + \dots + n^3 = \frac{n^2(n+1)^2}{4}$$

$$1^4 + 2^4 + 3^4 + \dots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

### 2.6 级数

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots, (-\infty < x < \infty)$$

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \dots, (-1 < x \leq 1)$$

$$\sqrt{1+x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \dots, (-1 \leq x \leq 1)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots, (-\infty < x < \infty)$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots, (-\infty < x < \infty)$$

### 2.7 概率论

设  $X$  离散随机变量, 取值为  $x$  的概率为  $p_X(x)$ . 期望值 (均值) 为  $\mu = \mathbb{E}(X) = \sum_x x p_X(x)$ , 方差为  $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$  其中  $\sigma$  是标准差. 如果  $X$  是连续随机变量, 它将具有概率密度函数  $f_X(x)$ , 上述求和将变为积分,  $p_X(x)$  被  $f_X(x)$  替换.

期望值具有线性性:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

对于相互独立的  $X$  和  $Y$ ,

$$V(aX + bY) = a^2 V(X) + b^2 V(Y).$$

#### 2.7.1 离散分布

##### 二项分布

在  $n$  次相互独立的是/否实验中, 每次实验以概率  $p$  成功, 则成功次数服从  $\text{Bin}(n, p)$ ,  $n = 1, 2, \dots$ ,  $0 \leq p \leq 1$ .

$$p(k) = \binom{n}{k} p^k (1-p)^{n-k}$$

$$\mu = np, \sigma^2 = np(1-p)$$

当  $p$  较小时,  $\text{Bin}(n, p)$  可近似为  $\text{Po}(np)$ .

##### 首次成功分布

在  $n$  次相互独立的是/否实验中, 每次实验以概率  $p$  成功, 则首次成功所需的试验次数服从  $\text{Fs}(p)$ ,  $0 \leq p \leq 1$ .

$$p(k) = p(1-p)^{k-1}, k = 1, 2, \dots$$

$$\mu = \frac{1}{p}, \sigma^2 = \frac{1-p}{p^2}$$

### 泊松分布

如果事件以已知平均速率  $\kappa$  发生，并且与距离上一次事件发生的时间相互独立，那么固定时间段  $t$  内发生的事件数服从  $\text{Po}(\lambda)$ ,  $\lambda = t\kappa$ .

$$p(k) = e^{-\lambda} \frac{\lambda^k}{k!}, k = 0, 1, 2, \dots$$

$$\mu = \lambda, \sigma^2 = \lambda$$

## Data structures (3)

UnionFind.h  
**Description:** 并查集数据结构。0-index.  
**Time:**  $\mathcal{O}(\alpha(N))$

2d91ed, 14 lines

```
struct UF {
    vi p;
    UF(int n) : p(n, -1) {}
    int find(int x) { return p[x] < 0 ? x : p[x] = find(p[x]); }
    bool merge(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (p[a] > p[b]) swap(a, b);
        p[a] += p[b]; p[b] = a;
        return true;
    }
    bool sameSet(int a, int b) { return find(a) == find(b); }
    int size(int x) { return -p[find(x)]; }
};
```

### wUF

d -= dt[b] - dt[a];

$$d = w[b] - w[a]$$

$$dt[b] = w[b] - w[rb]$$

$$dt[a] = w[a] - w[ra]$$

$$dt[rb] = w[rb] - w[ra]$$

$$= w[b] - dt[b] + dt[a] - w[a]$$

$$= d - (dt[b] - dt[a])$$

#### WeightedUnionFind.h

**Description:** 带权并查集数据结构，0-index. find(x) 后 dt[x] 表示 weight(x) - weight(root)。merge(a, b, d) 添加约束 weight(b) - weight(a) = d. 若与已有约束矛盾则返回 false. dist(a, b) 返回 weight(b) - weight(a), 即 dt[b] - dt[a], 要求 a 和 b 已连通。适用于有符号的加法类型，且不会溢出。  
**Time:**  $\mathcal{O}(\alpha(N))$

67b617, 24 lines

```
struct wUF {
    vi p, dt;
    wUF(int n) : p(n, -1), dt(n, 0) {}
    int find(int x) {
        if (p[x] < 0) return x;
        int rt = find(p[x]);
        dt[x] += dt[p[x]];
        return p[x] = rt;
    }
    bool merge(int a, int b, int d) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return dt[b] - dt[a] == d;
```

```
        d -= dt[b] - dt[a];
        if (p[ra] > p[rb]) swap(ra, rb), d = -d;
        p[ra] += p[rb]; p[rb] = ra; dt[rb] = d;
        return true;
    }
    int dist(int a, int b) {
        assert(sameSet(a, b));
        return dt[b] - dt[a];
    }
    bool sameSet(int a, int b) { return find(a) == find(b); }
    int size(int x) { return -p[find(x)]; }
};
```

## RMQ

$$f : \{(i, j) \in \mathbb{Z}^2 \mid 1 \leq i \leq n, \, 0 \leq j \leq \lfloor \log_2 n \rfloor\} \rightarrow \mathbb{R}$$

$$f(i, j) := \min\{a_k \mid k \in [i, i + 2^j], k \in \mathbb{Z}\}$$

显然 $f(i, 0) = a_i$

利用倍增，状态转移方程：

$$f(i, j) = \min \left( f(i, j - 1), f(i + 2^{j-1}, j - 1) \right)$$

$$\overbrace{a_i, \cdots, a_{i+2^j-1-1}, a_{i+2^j-1}, \cdots, a_{i+2^j-1}}^{f(i, j)}$$

查询 $[l, r)$ ：

$$\text{dep} = \lfloor \log_2(r - l) \rfloor = 31 - \_\_\text{builtin\_clz}(r - l)$$

$$\min\{a_k \mid k \in [l, r)\} = \min \left( f(l, \text{dep}), f(r - 2^{\text{dep}}, \text{dep}) \right)$$

$$\overbrace{a_l, \cdots, a_{r-2^{\text{dep}}}, \cdots, a_{l+2^{\text{dep}}-1}, \cdots, a_{r-1}}^{f(r-2^{\text{dep}}, \text{dep})}$$

$$\underbrace{\hspace{10em}}_{f(l, \text{dep})}$$

$$\hspace{10em} [l, r)$$

#### RMQ.h

**Description:** 区间最小值查询。返回 min(V[a], V[a + 1], ... V[b - 1])。  
**Usage:** RMQ rmq(values);  
**rmq.query**(inclusive, exclusive);  
**Time:**  $\mathcal{O}(|V| \log |V| + Q)$

b6e9e5, 15 lines

```
template<class T> struct RMQ {
    vector<vector<T>> > tbl;
    RMQ(const vector<T> &val) { // pw = 2^{k-1}
        for (int pw = 1, k = 1; pw * 2 <= sz(val); pw *= 2, ++k) {
            tbl.emplace_back(sz(val) - pw * 2 + 1);
            rep(j, 0, sz(tbl[k]))
                tbl[k][j] = min(tbl[k - 1][j], tbl[k - 1][j + pw]);
        }
    }
    T query(int a, int b) {
```

```
        assert(a < b); // or return inf if a == b
        int dep = 31 - __builtin_clz(b - a);
        return min(tbl[dep][a], tbl[dep][b - (1 << dep)]);
    }
};
```

#### FenwickTree.h

**Description:** 树状数组。计算前缀和 a[0] + a[1] + ... + a[pos - 1], 并通过计算新旧值之差更新单个元素 a[i]。  
**Time:** Both operations are  $\mathcal{O}(\log N)$ .

219f0b, 21 lines

```
struct FT {
    vector<ll> s;
    FT(int n) : s(n) {}
    void update(int pos, ll dif) { // a[pos] += dif
        for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
    }
    ll query(int pos) { // sum of values in [0, pos)
        ll res = 0;
        for (; pos > 0; pos &= pos - 1) res += s[pos-1];
        return res;
    }
    int lower_bound(ll sum) { // min pos st sum of [0, pos] >= sum
        // Returns n if no sum is >= sum, or -1 if empty sum is.
        if (sum <= 0) return -1;
        int pos = 0;
        for (int pw = 1 << 25; pw; pw >>= 1)
            if (pos + pw <= sz(s) && s[pos + pw - 1] < sum)
                pos += pw, sum -= s[pos-1];
        return pos;
    }
};
```

#### SubMatrix.h

**Description:** 在给定左上角和右下角的情况下，快速计算子矩阵和。坐标采用左闭右开区间。  
**Usage:** SubMatrix<int> m(matrix);  
**m.sum**(0, 0, 2, 2); // 左上角四个元素  
**Time:**  $\mathcal{O}(N^2 + Q)$

c59ada, 12 lines

```
template<class T> struct SubMatrix {
    vector<vector<T>> > p;
    SubMatrix(vector<vector<T>>& v) {
        int R = sz(v), C = sz(v[0]);
        p.assign(R+1, vector<T>(C+1));
        rep(r, 0, R) rep(c, 0, C)
            p[r+1][c+1] = v[r][c] + p[r][c+1] + p[r+1][c] - p[r][c];
    }
    T sum(int u, int l, int d, int r) {
        return p[d][r] - p[d][l] - p[u][r] + p[u][l];
    }
};
```

#### Matrix.h

**Description:** 方阵乘法、乘列向量、快速幂。默认无取模。  
**Usage:** Matrix<int, 3> A;  
A.d = {{{{1,2,3}}, {{4,5,6}}, {{7,8,9}}}};  
**array**<int, 3> vec = {1,2,3};  
**vec** = (A^N) \* vec;

9fd6b0, 23 lines

```
template<class T, int N> struct Matrix {
    using M = Matrix;
    array<array<T, N>, N> d{};
    M operator*(const M& m) const {
        M a;
        rep(i, 0, N) rep(j, 0, N)
            rep(k, 0, N) a.d[i][k] += d[i][j] * m.d[j][k];
        return a;
    }
};
```

```
array<T, N> operator*(const array<T, N>& vec) const {
    array<T, N> ret{};
    rep(i,0,N) rep(j,0,N) ret[i] += d[i][j] * vec[j];
    return ret;
}
M operator^(ll p) const {
    assert(p >= 0);
    M a, b(*this);
    rep(i,0,N) a.d[i][i] = 1;
    for (; p; b = b * b, p >>= 1)
        if (p & 1) a = a * b;
    return a;
}
};
```

### TagInfo.h

**Description:** 线段树节点 Info 与懒标记 Tag 的示例。

**Time:**  $\mathcal{O}(1)$ .

68f3e2, 17 lines

```
struct Tag {
    ll mul = 1, add = 0;
    void update(const Tag &t, int l, int r) {
        mul = (mul * t.mul) % MOD;
        add = (add * t.mul) % MOD;
        add = (add + t.add) % MOD;
    }
};
struct Info {
    ll val = 0;
    void update(const Tag &t, int l, int r) {
        val = (val * t.mul + t.add * (r - l)) % MOD;
    }
    static Info merge(const Info &a, const Info &b) {
        return {(a.val + b.val) % MOD};
    }
};
```

### SegmentTree.h

**Description:** 0-indexed 线段树，左闭右开区间。节点为 Info，具有默认构造，合并函数 static Info merge(const Info&, const Info&)。

**Time:**  $\mathcal{O}(\log N)$

42c79a, 20 lines

```
template<typename Info>struct SegmentTree {
    int n;vector<Info> seg;
    SegmentTree(const vector<Info> &val): n(sz(val)), seg(2 * n) {
        rep(i, 0, n) seg[n + i] = val[i];
        for (int i = n - 1; i > 0; --i)
            seg[i] = Info::merge(seg[i << 1], seg[i << 1 | 1]);
    }
    void update(int pos, const Info &val) {
        for (seg[pos += n] = val; pos >>= 1;)
            seg[pos] = Info::merge(seg[pos << 1], seg[pos << 1 | 1]);
    }
    Info query(int l, int r) { // query [l, r)
        Info resl {}, resr {};
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l & 1) resl = Info::merge(resl, seg[l++]);
            if (r & 1) resr = Info::merge(seg[--r], resr);
        }
        return Info::merge(resl, resr);
    }
};
```

### LazySegmentTreeNew.h

**Description:** 通用懒标记线段树，使用外部定义的 Info 和 Tag。

**Time:**  $\mathcal{O}(\log N)$ .

"TagInfo.h"

a2cdb3, 45 lines

```
template<typename Info, typename Tag> struct Seg {
    vector<Info> seg;vector<Tag> tag;
```

```
Seg(const vector<Info> &val):seg(sz(val)*4), tag(sz(val)*4) {
    auto build = [&](int l, int r, int root, auto &&self) {
        if (r - l == 1) {
            seg[root] = val[l];
            return;
        }
        int mid = (l + r) >> 1;
        self(l, mid, root << 1, self);
        self(mid, r, root << 1 | 1, self);
        seg[root]=Info::merge(seg[root << 1],seg[root << 1 | 1]);
    };
    build(0, sz(val), 1, build);
}
void push(int l, int r, int root) {
    int m = (l + r) >> 1;
    seg[root << 1].update(tag[root], l, m);
    tag[root << 1].update(tag[root], l, m);
    seg[root << 1 | 1].update(tag[root], m, r);
    tag[root << 1 | 1].update(tag[root], m, r);
    tag[root] = Tag();
}
void update(int l,int r,int ql,int qr,int root,const Tag &t){
    if (r <= ql || qr <= l) return;
    if (ql <= l && r <= qr) {
        seg[root].update(t, l, r);
        tag[root].update(t, l, r);
        return;
    }
    push(l, r, root);
    int m = (l + r) >> 1;
    update(l, m, ql, qr, root << 1, t);
    update(m, r, ql, qr, root << 1 | 1, t);
    seg[root] = Info::merge(seg[root << 1],seg[root << 1 | 1]);
}
Info query(int l, int r, int ql, int qr, int root) {
    if (r <= ql || qr <= l) return Info();
    if (ql <= l && r <= qr) return seg[root];
    push(l, r, root);
    int m = (l + r) >> 1;
    return Info::merge(query(l, m, ql, qr, root << 1),
        query(m, r, ql, qr, root << 1 | 1));
}
};
```

### Treap.h

**Description:** 一种简短的自平衡树。顺序容器，支持对数时间 split/merge，并且容易扩展额外信息。

Usage: rt = tr.merge(rt, tr.add(val));

rt = tr.insert(rt, tr.add(val), pos);

tr.each(rt, [](int x) {cout << x << ' ' ;});

**Time:**  $\mathcal{O}(\log N)$

41397d, 54 lines

```
struct Treap {
    struct Node {
        int l = 0, r = 0;
        int val = 0, pr = 0, siz = 1;
        Node(int val) : val(val), pr(rand()) {}
    };
    vector<Node> adj{Node(0)};
    int size(int x) { return x ? adj[x].siz : 0; }
    void recalsiz(int x) {
        if (x) adj[x].siz = size(adj[x].l) + size(adj[x].r) + 1;
    }
    int add(int val) {
        adj.push_back(Node(val));
        return sz(adj) - 1;
    }
    template<class F> void each(int x, F f) {
        if (x) each(adj[x].l, f), f(adj[x].val), each(adj[x].r, f);
```

```
}
pair<int, int> split(int x, int k) {
    if (!x) return {0, 0};
    if (size(adj[x].l)>=k) { // "adj[x].val>=k" for lower_bound(k)
        auto [L,R] = split(adj[x].l, k);
        adj[x].l = R;
        recalsiz(x);
        return {L, x};
    } else { // and just "k"
        auto [L,R] = split(adj[x].r, k - size(adj[x].l) - 1);
        adj[x].r = L;
        recalsiz(x);
        return {x, R};
    }
}
int merge(int l, int r) {
    if (!l || !r) return l | r;
    if (adj[l].pr > adj[r].pr) {
        adj[l].r = merge(adj[l].r, r);
        return recalsiz(l), l;
    } else {
        adj[r].l = merge(l, adj[r].l);
        return recalsiz(r), r;
    }
}
int insert(int rt, int x, int pos) {
    auto [l,r] = split(rt, pos);
    return merge(merge(l, x), r);
}
// Example application: move the range [l, r) to index k
void move(int& rt, int l, int r, int k) {
    int a, b, c;
    tie(a,b) = split(rt, l); tie(b,c) = split(b, r - l);
    if (k <= l) rt = merge(insert(a, b, k), c);
    else rt = merge(a, insert(c, b, k - r));
}
};
```

## Numerical (4)

### 4.1 Optimization

#### GoldenSectionSearch.h

**Description:** 在区间  $[a, b]$  中寻找使得  $f(x)$  最小化的  $x$ ，假设  $f$  在该区间上是单峰的，即只有一个局部最小值且没有局部最大值。结果的最大误差为  $eps$ 。对于最大化问题，只需在代码中稍作修改即可。离散版本参见 Various 章节中的 TernarySearch.h。

Usage: double func(double x) { return 4+x+.3\*x\*x; }

double xmin = gss(-1000,1000,func);

**Time:**  $\mathcal{O}(\log((b-a)/\epsilon))$

d7b114, 15 lines

```
template<class F>
double gss(double a, double b, F f) {
    double r = (sqrt(5)-1)/2, eps = 1e-7;
    double x1 = b - r*(b-a), x2 = a + r*(b-a);
    double f1 = f(x1), f2 = f(x2);
    while (b-a > eps)
        if (f1 < f2) { // 改为 f1 > f2 则为最大化
            b = x2; x2 = x1; f2 = f1;
            x1 = b - r*(b-a); f1 = f(x1);
        } else {
            a = x1; x1 = x2; f1 = f2;
            x2 = a + r*(b-a); f2 = f(x2);
        }
    return a;
}
```



## Number theory (5)

### 5.1 整除性 欧几里得

$$\gcd(a, b) = 1$$

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

找符合 $ax + by = d = \gcd(a, b)$ 的一组 $x, y$

$$a \cdot a_{\text{inv}} + py = 1 = \gcd(a, p)$$

$$by + (a \bmod b)x = d$$

$$ax + b(y - \left\lfloor \frac{a}{b} \right\rfloor x) = d$$

euclid.h

**Description:** 找到两个整数  $x$  和  $y$ , 使得  $ax + by = \gcd(a, b)$ 。如果你只需要  $\gcd$ , 可以使用内置的 `__gcd`。如果  $a$  和  $b$  互质, 那么  $x$  就是  $a \pmod b$  的逆元。  
**Time:**  $\mathcal{O}(\log \min(a, b))$

33ba8f, 5 lines

```
ll euclid(ll a, ll b, ll &x, ll &y) {
    if (!b) return x = 1, y = 0, a;
    ll d = euclid(b, a % b, y, x);
    return y -= a/b * x, d;
}
```

### 中国剩余定理

定理 CRT. 设  $m_1, m_2, \dots, m_n$  两两互质, 且

$$(S): \begin{cases} x \equiv a_1 \pmod{m_1}, \\ x \equiv a_2 \pmod{m_2}, \\ \vdots \\ x \equiv a_n \pmod{m_n}. \end{cases}$$

令

$$M = \prod_{i=1}^n m_i, \quad M_i = \frac{M}{m_i}, \quad t_i \equiv M_i^{-1} \pmod{m_i}.$$

则方程组  $(S)$  有解, 并且一组解为

$$x \equiv \sum_{i=1}^n a_i t_i M_i \pmod{M}.$$

**构造性证明.** 由于  $m_1, m_2, \dots, m_n$  两两互质, 对任意  $i$ , 有

$$\gcd(M_i, m_i) = 1.$$

因此存在  $t_i$ , 使得

$$t_i M_i \equiv 1 \pmod{m_i}.$$

令

$$x = \sum_{k=1}^n a_k t_k M_k.$$

固定任意  $i$ . 当  $k \neq i$  时, 由于  $m_i \mid M_k$ , 有

$$a_k t_k M_k \equiv 0 \pmod{m_i}.$$

于是

$$\begin{aligned} x &= a_i t_i M_i + \sum_{k \neq i} a_k t_k M_k \\ &\equiv a_i \cdot 1 + \sum_{k \neq i} 0 \pmod{m_i} \\ &\equiv a_i \pmod{m_i}. \end{aligned}$$

因此  $x$  同时满足所有同余式, 故  $x$  是方程组  $(S)$  的一组解。

CRT.h

**Description:** 中国剩余定理。  
**crt(a, m, b, n)** 计算  $x$ , 使得  $x \equiv a \pmod m, x \equiv b \pmod n$ 。如果  $|a| < m$  且  $|b| < n$ , 则  $x$  将满足  $0 \leq x < \text{lcm}(m, n)$ 。假设  $mn < 2^{62}$ 。

**Time:**  $\log(n)$

"euclid.h" 04d93a, 7 lines

```
ll crt(ll a, ll m, ll b, ll n) {
    if (n > m) swap(a, b), swap(m, n);
    ll x, y, g = euclid(m, n, x, y);
    assert((a - b) % g == 0); // else no solution
    x = (b - a) % n * x % n / g * m + a;
    return x < 0 ? x + m*n/g : x;
}
```

### 5.2 模运算

ModularArithmetic.h

**Description:** 用于模运算, 设置 MOD 后使用。

"euclid.h" 2f9ed0, 18 lines

```
constexpr ll MOD = 17; // change to something else
struct Mod {
    ll x;
    Mod(ll _x) : x((_x % MOD + MOD) % MOD) {}
    Mod operator+(Mod b) { return Mod(((x + b.x)%MOD+MOD)%MOD); }
    Mod operator-(Mod b) { return Mod(((x - b.x)%MOD+MOD)%MOD); }
    Mod operator*(Mod b) { return Mod((x * b.x %MOD+MOD)%MOD); }
    Mod operator/(Mod b) { return *this * invert(b); }
    Mod invert(Mod a) {
        ll x, y, g = euclid(a.x, MOD, x, y);
        assert(g == 1); return Mod((x % MOD + MOD) % MOD);
    }
    Mod operator^(ll e) {
        if (!e) return Mod(1);
        Mod r = *this ^ (e / 2); r = r * r;
        return e & 1 ? *this * r : r;
    }
};
```

### 模逆元

$$\frac{a}{b} \equiv a \cdot b_{\text{inv}} \equiv x \pmod{p}$$

$$\Rightarrow a \equiv bx \pmod{p}$$

$$\Rightarrow a \cdot b_{\text{inv}} \cdot b \equiv bx \pmod{p}$$

$$\Rightarrow a \cdot (b_{\text{inv}} \cdot b - 1) \equiv 0 \pmod{p}$$

$$\Rightarrow b_{\text{inv}} \cdot b \equiv 1 \pmod{p}$$

$$\text{inv存在} \Leftrightarrow \gcd(b, p) = 1$$

$$b \cdot b_{\text{inv}} = k \cdot p + 1, \quad g = \gcd(b, p) \Rightarrow g \cdot b_1 \cdot b_{\text{inv}} = k \cdot g \cdot p_1 + 1$$

$$\Rightarrow g \cdot (b_1 \cdot b_{\text{inv}} - k \cdot p_1) = 1$$

$$\text{数域为 } \mathbb{Z} \Rightarrow g = 1, \quad b_1 \cdot b_{\text{inv}} - k \cdot p_1 = 1$$

### 线性求逆元

$p$  为质数,  $p > n$

$$\forall x \in \mathbb{N}^+, \quad p = kx + r, \quad k = \left\lfloor \frac{p}{x} \right\rfloor, \quad r = p \bmod x$$

$$\begin{aligned} kx + r \equiv 0 \pmod{p} &\Rightarrow kx \equiv -r \pmod{p} \\ &\Rightarrow k \cdot r_{\text{inv}} \cdot x \equiv -1 \pmod{p} \end{aligned}$$

$$\begin{aligned} x \cdot x_{\text{inv}} \equiv 1 \pmod{p} &\Rightarrow k \cdot r_{\text{inv}} \cdot x \equiv -x \cdot x_{\text{inv}} \pmod{p} \\ &\Rightarrow x_{\text{inv}} \equiv -k \cdot r_{\text{inv}} \pmod{p} \end{aligned}$$

$$r = p \bmod x < x \Rightarrow r_{\text{inv}} \text{ 比 } x_{\text{inv}} \text{ 先被求出}$$

ModInverse.h

**Description:** 预计算模逆元, 假设  $\text{LIM} \leq \text{mod}$  且  $\text{mod}$  是质数。

17b468, 4 lines

```
constexpr ll MOD = 1000000007, LIM = 200000;
vector<int> inv(LIM + 1, 0); inv[1] = 1;
for (int i = 2; i <= LIM; i++)
    inv[i] = MOD - (MOD / i) * inv[MOD % i] % MOD;
```

ModPow.h

e33b0b, 8 lines

constexpr ll MOD = 1000000007; // faster if const

```
ll modpow(ll b, ll e) {
    ll ans = 1;
    for (; e; b = b * b % MOD, e /= 2)
        if (e & 1) ans = ans * b % MOD;
    return ans;
}
```

ModMulLL.h

**Description:** Calculate  $a \cdot b \bmod c$  (or  $a^b \bmod c$ ) for  $0 \leq a, b \leq c \leq 7.2 \cdot 10^{18}$ .

**Time:**  $\mathcal{O}(1)$  for `modmul`,  $\mathcal{O}(\log b)$  for `modpow`

b2cd62, 27 lines

using ull = unsigned long long;

```
// // 浮点数估计商, 整数修正余数, 快, 需要long double至少有64位有效精度
static_assert(std::numeric_limits<long double>::digits >= 64,
    "long double precision is not enough for this
    modmul");
ull modmul(ull a, ull b, ull mod) {
    ll ret = a * b - mod * ull(1.L / mod * a * b);
    return ret + mod * (ret < 0) - mod * (ret >= (ll) mod);
}
// O(log),慢
ull modmul(ull x, ull y, ull mod) {
    ull res = 0;
    for (res %= mod; y; y >= 1, x += x >= mod - x ? x - mod : x)
        if (y & 1) res += res >= mod - x ? x - mod : x;
    return res;
}
```

```
// 需要__uint128_t
ull modmul(ull a, ull b, ull mod) {
    return (__uint128_t) a * b % mod;
}
ull modpow(ull b, ull e, ull mod) {
    ull ans = 1;
    for (; e; b = modmul(b, b, mod), e /= 2)
        if (e & 1) ans = modmul(ans, b, mod);
    return ans;
}
```

### 5.3 Primality

Eratosthenes.h

**Description:** 生成不超过给定上界的所有质数的质数筛。isprime[i] 为真当且仅当 i 是质数。

**Time:**  $\lim=100'000'000 \approx 0.8$  s. Runs 30% faster if only odd indices are stored.

bfb4ed, 11 lines

```
constexpr int pmx = 5'000'000;
bitset<pmx> isp;
vi eratosthenesSieve(int lim) {
    isp.set(); isp[0] = isp[1] = 0;
    for (int i = 4; i < lim; i += 2) isp[i] = 0;
    for (int i = 3; i*i < lim; i += 2) if (isp[i])
        for (int j = i*i; j < lim; j += i*2) isp[j] = 0;
    vi ret;
    rep(i, 2, lim) if (isp[i]) ret.push_back(i);
    return ret;
}
```

## MillerRabin.h

**Description:** 确定性的 Miller-Rabin 素性测试。保证对不超过  $7 \cdot 10^{18}$  的数有效；更大的数建议使用 Python 并随机扩展测试底数集合 A。

**Time:** 7 times the complexity of  $a^b \bmod c$ .

"ModMuLL.h" 5cce9d, 12 lines

```
bool isPrime(ull n) {
    if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
    vector<ull> A = {2, 325, 9375, 28178, 450775, 9780504, 1795265022};
    ull s = __builtin_ctzll(n - 1), d = n >> s;
    for (ull a : A) {
        ull p = modpow(a % n, d, n), i = s;
        while (p != 1 && p != n - 1 && a % n && i--)
            p = modmul(p, p, n);
        if (p != n - 1 && i != s) return false;
    }
    return true;
}
```

## Factor.h

**Description:** Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).

**Time:**  $\mathcal{O}(n^{1/4})$ , less for numbers with small factors.

"ModMuLL.h", "MillerRabin.h" d8d98d, 18 lines

```
ull pollard(ull n) {
    ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
    auto f = [&](ull x) { return modmul(x, x, n) + i; };
    while (t++ % 40 || __gcd(prd, n) == 1) {
        if (x == y) x = ++i, y = f(x);
        if ((q = modmul(prd, max(x, y) - min(x, y), n))) prd = q;
        x = f(x), y = f(f(y));
    }
    return __gcd(prd, n);
}
vector<ull> factor(ull n) {
    if (n == 1) return {};
    if (isPrime(n)) return {n};
    ull x = pollard(n);
    auto l = factor(x), r = factor(n / x);
    l.insert(l.end(), all(r));
    return l;
}
```

# Graph (6)

## 6.1 Fundamentals

### BellmanFord.h

**Description:** 计算可能含负权边的图中从 s 出发的最短路。不可达点的距离为 inf；能经由负权环到达的点距离为 -inf。假设  $V^2 \max |w_i| < \sim 2^{63}$ 。

**Time:**  $\mathcal{O}(VE)$

830a8f, 23 lines

```
const ll inf = LLONG_MAX;
```

```
struct Ed { int a, b, w, s() { return a < b ? a : -a; } };
struct Node { ll dist = inf; int prev = -1; };
```

```
void bellmanFord(vector<Node>& nodes, vector<Ed>& eds, int s) {
    nodes[s].dist = 0;
    sort(all(eds), [](Ed a, Ed b) { return a.s() < b.s(); });
```

```
    int lim = sz(nodes) / 2 + 2; // /3+100 with shuffled vertices
    rep(i, 0, lim) for (Ed ed : eds) {
        Node cur = nodes[ed.a], &dest = nodes[ed.b];
        if (abs(cur.dist) == inf) continue;
        ll d = cur.dist + ed.w;
        if (d < dest.dist) {
            dest.prev = ed.a;
            dest.dist = (i < lim-1 ? d : -inf);
        }
    }
    rep(i, 0, lim) for (Ed e : eds) {
        if (nodes[e.a].dist == -inf)
            nodes[e.b].dist = -inf;
    }
}
```

## TopoSort.h

**Description:** 拓扑排序。给定一个有向图，输出一个顶点排列，使得所有边都从排列左侧指向右侧。如果图中存在环，返回列表的大小会小于 n，从环可达的点不会被返回。

**Time:**  $\mathcal{O}(|V| + |E|)$

4bbc3a, 8 lines

```
vi topoSort(const vector<vi>& adj) {
    vi indeg(sz(adj)), tfn;
    for (auto& li : adj) for (int v : li) indeg[v]++;
    rep(i, 0, sz(adj)) if (indeg[i] == 0) tfn.push_back(i);
    rep(j, 0, sz(tfn)) for (int v : adj[tfn[j]])
        if (--indeg[v] == 0) tfn.push_back(v);
    return tfn;
}
```

## 6.2 Trees

### BinaryLifting.h

**Description:** 计算树上 2 的幂次跳跃，以支持快速向上跳跃和 LCA。根节点指向自己。

**Time:** construction  $\mathcal{O}(N \log N)$ , queries  $\mathcal{O}(\log N)$

3f4678, 25 lines

```
vector<vi> treeJump(vi& fa) {
    int pw = 1, k = 1;
    while(pw < sz(fa)) pw *= 2, k++;
    vector<vi> tbl(k, fa);
    rep(i, 1, k) rep(j, 0, sz(fa))
        tbl[i][j] = tbl[i-1][tbl[i-1][j]];
    return tbl;
}
```

```
int jmp(vector<vi>& tbl, int nod, int steps) {
    rep(i, 0, sz(tbl))
        if (steps & (1<<i)) nod = tbl[i][nod];
    return nod;
}
```

```
int lca(vector<vi>& tbl, vi& depth, int a, int b) {
    if (depth[a] < depth[b]) swap(a, b);
    a = jmp(tbl, a, depth[a] - depth[b]);
    if (a == b) return a;
    for (int i = sz(tbl); i--;) {
        int pa = tbl[i][a], pb = tbl[i][b];
        if (pa != pb) a = pa, b = pb;
    }
    return tbl[0][a];
}
```

## LCA.h

**Description:** 计算树的 LCA 的数据结构。树以 1 为根。C 为树的邻接表，可以有向可以无向。

**Time:**  $\mathcal{O}(N \log N + Q)$

"/data-structures/RMQ.h" cb8d4c, 21 lines

```
struct LCA {
    int T = 0;
    vi time, path, ret;
    RMQ<int> rmq;

    LCA(vector<vi> &adj):time(sz(adj)),rmq((dfs(adj,1,-1),ret)){}
    void dfs(vector<vi> &adj, int u, int fa) {
        time[u] = T++;
        for (int v : adj[u]) if (v != fa) {
            path.push_back(u), ret.push_back(time[u]);
            dfs(adj, v, u);
        } // tout[u] = T; size_of_subtree[u] = tout[u] - time[u];
    }
}
```

```
int lca(int a, int b) {
    if (a == b) return a;
    tie(a, b) = minmax(time[a], time[b]);
    return path[rmq.query(a, b)];
}
//dist(a,b){return depth[a] + depth[b] - 2*depth[lca(a,b)];}
```

## CompressTree.h

**Description:** 给定一棵有根树和一个非空节点子集 S，计算包含所有节点的最小子树，通过添加所有 (最多  $|S| - 1$ ) 相邻节点对的 LCA 并压缩边。返回一个 (par, orig\_index) 的列表，根节点为 ret[0]。根节点指向自己。

**Time:**  $\mathcal{O}(|S| \log |S|)$

"LCA.h" 63e21d, 21 lines

```
using vpi = vector<pair<int, int> >;
vpi compressTree(LCA& lca, const vi& subset) {
    static vi rev; rev.resize(sz(lca.time));
    vi li = subset, &T = lca.time;
    auto cmp = [&](int a, int b) { return T[a] < T[b]; };
    sort(all(li), cmp);
    int m = sz(li)-1;
    rep(i, 0, m) {
        int a = li[i], b = li[i+1];
        li.push_back(lca.lca(a, b));
    }
    sort(all(li), cmp);
    li.erase(unique(all(li)), li.end());
    rep(i, 0, sz(li)) rev[li[i]] = i;
    vpi ret = {pii(0, li[0])};
    rep(i, 0, sz(li)-1) {
        int a = li[i], b = li[i+1];
        ret.emplace_back(rev[lca.lca(a, b)], b);
    }
    return ret;
}
```

# Geometry (7)

## 7.1 Geometric primitives

CW ::= ClockWise 顺时针

CCW ::= CounterClockWise 逆时针

## Point.h

**Description:** 平面点类。T 为 double 或 long long 等类型 (避免 int)

dc820b, 30 lines

```
constexpr double eps = 0;
template<class T> int sgn(T x) { return (x > eps)-(x < -eps); }
```

```
template<class T> struct Point {
    using P = Point;
    T x, y;
    explicit Point(T x = 0, T y = 0) : x(x), y(y) {}
    bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
    bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
    bool approxEq(P p) const { return !(sgn(x-p.x)|sgn(y-p.y)); }
    P operator+(P p) const { return P(x + p.x, y + p.y); }
    P operator-(P p) const { return P(x - p.x, y - p.y); }
    P operator*(T d) const { return P(x * d, y * d); }
    P operator/(T d) const { return P(x / d, y / d); }
    T dot(P p) const { return x * p.x + y * p.y; } // 点乘
    T cross(P p) const { return x * p.y - y * p.x; } // 叉乘
    // Oa与Ob叉乘
    T cross(P a, P b) const { return (a-*this).cross(b-*this); }
    T dist2() const { return x * x + y * y; } // 到原点距离平方
    double dist()const{return sqrt((double)dist2());} //到原点距离
    // 与x轴夹角, 范围[-pi,pi]
    double angle() const { return atan2(y, x); }
    P unit() const { return *this/dist(); } // 单位化
    P perp() const { return P(-y, x); } // 绕原点逆时针旋转九十度
    P normal() const { return perp().unit(); } // 法向量
    // 绕原点逆时针旋转a弧度
    P rotate(double a) const {
        return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
    friend ostream& operator<<(ostream& os, P p) {
        return os << "(" << p.x << ", " << p.y << ")"; }
};
```

$$dis = \frac{|(e-s) \times (p-s)|}{|e-s|}$$

$$dis \leq eps \Leftrightarrow |(e-s) \times (p-s)| \leq |e-s| \cdot eps$$

### sideOf.h

**Description:** 返回点  $p$  相对于线段  $se$  的位置。  $1/0/-1 \Leftrightarrow$  左侧/在线上/右侧。如果给定了可选参数  $eps$ , 则当点  $p$  距离线段  $se$  的距离小于  $eps$  时, 返回 0。  $P$  应该是  $\text{Point}<T>$  类型, 其中  $T$  可以是  $\text{double}$  或  $\text{long long}$  等类型。在中间步骤中使用了乘积运算, 因此如果使用  $\text{int}$  或  $\text{long long}$  类型, 请注意溢出。

**Usage:** `bool left = sideOf(p1,p2,q)==1;`

"Point.h" 3af81c, 9 lines

```
template<class P>
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }
```

```
template<class P>
int sideOf(const P& s, const P& e, const P& p, double eps) {
    auto a = (e-s).cross(p-s); // 避免 s==e
    double l = (e-s).dist()*eps;
    return (a > l) - (a < -l);
}
```

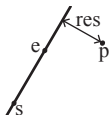
### lineDistance.h

#### Description:

返回点  $p$  到包含点  $a$  和  $b$  的直线的有符号距离。从  $a$  到  $b$  看, 左侧为正值, 右侧为负值。  $a==b$  时返回  $\text{nan}$ 。  $P$  应该是  $\text{Point}<T>$  或  $\text{Point3D}<T>$  类型, 其中  $T$  可以是  $\text{double}$  或  $\text{long long}$  等类型。在中间步骤中使用了乘积运算, 因此如果使用  $\text{int}$  或  $\text{long long}$  类型, 请注意溢出。使用  $\text{Point3D}$  将始终给出非负距离。对于  $\text{Point3D}$ , 请在叉积的结果上调用  $\text{dist}$ 。

"Point.h" f6bf6b, 4 lines

```
template<class P>
double lineDist(const P& a, const P& b, const P& p) {
    return (double)(b-a).cross(p-a)/(b-a).dist();
}
```



## 求直线交点

$1/0/-1 \Leftrightarrow$  唯一交点/无交点/重合

$$L_1 : s_1 + t(e_1 - s_1)$$

$$L_2 : s_2 + u(e_2 - s_2)$$

$$\text{交点 } X = s_1 + t(e_1 - s_1) = s_2 + u(e_2 - s_2)$$

$$\overrightarrow{s_2 X} \parallel \overrightarrow{s_2 e_2} \Rightarrow (X - s_2) \times (e_2 - s_2) = 0$$

$$\Rightarrow (s_1 + t(e_1 - s_1) - s_2) \times (e_2 - s_2) = 0$$

$$\Rightarrow (s_1 - s_2) \times (e_2 - s_2) + t(e_1 - s_1) \times (e_2 - s_2) = 0$$

$$\Rightarrow t = \frac{(e_2 - s_2) \times (s_1 - s_2)}{(e_1 - s_1) \times (e_2 - s_2)}$$

$$X = s_1 + \frac{(e_2 - s_2) \times (s_1 - s_2)}{(e_1 - s_1) \times (e_2 - s_2)} \cdot (e_1 - s_1)$$

$$= \{[(e_1 - s_1) \times (e_2 - s_2) - (e_2 - s_2) \times (s_1 - s_2)] \cdot s_1 + [(e_2 - s_2) \times (s_1 - s_2)] \cdot e_1\} \cdot \frac{1}{(e_1 - s_1) \times (e_2 - s_2)}$$

$$= \{[(e_1 - s_1) \times (e_2 - s_2) + (s_1 - s_2) \times (e_2 - s_2)] \cdot s_1 + [(e_2 - s_2) \times (s_1 - s_2)] \cdot e_1\} \cdot \frac{1}{(e_1 - s_1) \times (e_2 - s_2)}$$

$$= \frac{[(e_1 - s_2) \times (e_2 - s_2)] \cdot s_1 + [(e_2 - s_2) \times (s_1 - s_2)] \cdot e_1}{(e_1 - s_1) \times (e_2 - s_2)}$$

$$= \frac{p \cdot s_1 + q \cdot e_1}{d}$$

### lineIntersection.h

#### Description:

如果存在唯一的交点, 返回  $\{1, \text{交点}\}$ ; 如果不存在交点, 返回  $\{0, (0,0)\}$ ; 如果存在无数个交点, 返回  $\{-1, (0,0)\}$ 。如果使用  $\text{Point}<\text{ll}>$ , 且交点坐标不全为整数, 则返回错误的结果。中间步骤使用了三个坐标的乘积, 因此如果使用  $\text{int}$  或  $\text{ll}$ , 请注意可能的溢出。

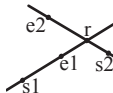
**Usage:** `auto res = lineInter(s1,e1,s2,e2);`

`if (res.first == 1)`

`cout << "intersection point at " << res.second << endl;`

"Point.h" a01f81, 8 lines

```
template<class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
    auto d = (e1 - s1).cross(e2 - s2);
    if (d == 0) // if parallel
        return {-(s1.cross(e1, s2) == 0), P(0, 0)};
    auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
    return {1, (s1 * p + e1 * q) / d};
}
```



## 点线距离

$$q \leftrightarrow = \text{proj}_{\overrightarrow{se}}(p)$$

$$(p - q \leftrightarrow) \cdot (e - s) = 0,$$

$$q \leftrightarrow = s + \lambda \leftrightarrow (e - s),$$

$$(p - s - \lambda \leftrightarrow (e - s)) \cdot (e - s) = 0,$$

$$(p - s) \cdot (e - s) = \lambda \leftrightarrow ((e - s) \cdot (e - s)),$$

$$\lambda \leftrightarrow = \frac{(p - s) \cdot (e - s)}{\|e - s\|^2},$$

$$\lambda_{\overline{se}} = \text{clamp}\left(\frac{(p - s) \cdot (e - s)}{\|e - s\|^2}, 0, 1\right),$$

$$q_{\overline{se}} = s + \lambda_{\overline{se}}(e - s),$$

$$\text{segDist}(s, e, p) = \|p - q_{\overline{se}}\|.$$

### SegmentDistance.h

#### Description:

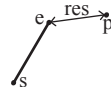
返回点  $p$  到线段  $se$  的最短距离。

**Usage:** `Point<double> a, b(2,2), p(1,1);`

`bool onSegment = segDist(a,b,p) < 1e-10;`

"Point.h" cbce89, 7 lines

```
using P = Point<double>;
double segDist(P &s, P &e, P &p) {
    if (s == e) return (p - s).dist();
    auto d = (e-s).dist2(), t = max(0, min(d, (p-s).dot(e-s)));
    // t = clamp((p - s).dot(e - s), 0, d);
    return ((p - s) * d - (e - s) * t).dist() / d;
}
```



## 点在线段上

$$(e - s) \times (p - s) = 0 \Leftrightarrow p \in \overleftrightarrow{se},$$

$$(p - s) \cdot (p - e) = 0 \Leftrightarrow p = s \text{ or } p = e,$$

$$(p - s) \cdot (p - e) < 0 \Leftrightarrow p \in \overline{se}^\circ.$$

### OnSegment.h

**Description:** 当且仅当  $p$  位于线段  $se$  上时返回  $\text{true}$ 。使用  $\text{Point}<\text{double}>$  时改用  $(\text{segDist}(s,e,p) \leq \text{epsilon})$ 。

"Point.h", "SegmentDistance.h" 7f1d77, 7 lines

```
template<class P> bool onSegment(P s, P e, P p) {
    return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

```
template<class P> bool onSegment(P s, P e, P p, double eps) {
    return segDist(s, e, p) <= eps;
}
```

## 线段交点

$$|res| = \begin{cases} 0, & \text{无交点,} \\ 1, & \text{唯一交点,} \\ 2, & \text{无限交点, 返回公共线段的两个端点.} \end{cases}$$



$$\begin{aligned}os_1 &= \overrightarrow{s_2 e_2} \times \overrightarrow{s_2 s_1}, \\oe_1 &= \overrightarrow{s_2 e_2} \times \overrightarrow{s_2 e_1}, \\os_2 &= \overrightarrow{s_1 e_1} \times \overrightarrow{s_1 s_2}, \\oe_2 &= \overrightarrow{s_1 e_1} \times \overrightarrow{s_1 e_2}.\end{aligned}$$

$$\begin{aligned}\text{唯一交点 } p &= s_1 + t(e_1 - s_1), \\(e_2 - s_2) \times (p - s_2) &= 0.\end{aligned}$$

$$\begin{aligned}0 &= (e_2 - s_2) \times (s_1 + t(e_1 - s_1) - s_2) \\&= (e_2 - s_2) \times (s_1 - s_2) + t(e_2 - s_2) \times (e_1 - s_1) \\&= (e_2 - s_2) \times (s_1 - s_2) + t(e_2 - s_2) \times ((e_1 - s_2) + (s_2 - s_1)) \\&= os_1 + t(oe_1 - os_1).\end{aligned}$$

$$t = \frac{os_1}{os_1 - oe_1}.$$

$$\begin{aligned}p &= s_1 + \frac{os_1}{os_1 - oe_1}(e_1 - s_1) \\&= \frac{s_1(os_1 - oe_1) + os_1(e_1 - s_1)}{os_1 - oe_1} \\&= \frac{s_1 oe_1 - e_1 os_1}{oe_1 - os_1}.\end{aligned}$$

### SegmentIntersection.h

#### Description:

若从 s1 到 e1 的线段与从 s2 到 e2 的线段存在唯一交点，则返回该交点。若不存在交点，则返回空 vector。若存在无限多个交点，则返回一个包含 2 个元素的 vector，其中存放公共线段的两个端点。如果 P 是 Point<ll> 且交点坐标不是整数，返回的位置会是错误的。中间步骤会用到三个坐标的乘积，因此使用 int 或 long long 时要注意溢出。

Usage: vector<P> inter = segInter(s1,e1,s2,e2);

if (sz(inter)==1)

cout << "segments intersect at " << inter[0] << endl;

"Point.h", "OnSegment.h" 4ae7a2, 13 lines

```
template<class P> vector<P> segInter(P s1, P e1, P s2, P e2) {
    auto os1 = s2.cross(e2, s1), oe1 = s2.cross(e2, e1),
        os2 = s1.cross(e1, s2), oe2 = s1.cross(e1, e2);
```

// 判断交点是否为唯一的非端点交点。

```
if (sgn(os1) * sgn(oe1) < 0 && sgn(os2) * sgn(oe2) < 0)
    return {(s1 * oe1 - e1 * os1) / (oe1 - os1)};
```

```
set<P> res;
```

```
if (onSegment(s2, e2, s1)) res.insert(s1);
```

```
if (onSegment(s2, e2, e1)) res.insert(e1);
```

```
if (onSegment(s1, e1, s2)) res.insert(s2);
```

```
if (onSegment(s1, e1, e2)) res.insert(e2);
```

```
return {all(res)};
```

```
}
```

### LineProjectionReflection.h

**Description:** 将点 p 投影到直线 ab 上。若设置 refl=true，则改为返回点 p 关于直线 ab 的对称点。如果 P 是整数点且目标点坐标不是整数，返回的点会是错误的。中间步骤会用到三个坐标的乘积，因此要注意溢出。

"Point.h" b5562d, 5 lines

```
template<class P>
P lineProj(P a, P b, P p, bool refl=false) {
    P v = b - a;
```

```
    return p - v.perp()*(1+refl)*v.cross(p-a)/v.dist2();
}
```

## 线性变换

找变换  $T$ ，使其满足  $T(p_0) = q_0$ ,  $T(p_1) = q_1$

$$T(x) = q_0 + A(x - p_0)$$

构造矩阵  $A$ ，使得  $A(p_1 - p_0) = q_1 - q_0$

$$\text{即 } A dp = dq$$

$$A = sR(\theta)$$

$$= \begin{bmatrix} s \cos \theta & -s \sin \theta \\ s \sin \theta & s \cos \theta \end{bmatrix}$$

$$\begin{aligned}|dq| &= |A dp| & dp \cdot dq &= |dp| |dq| \cos \theta \\&= |s \cdot dp| & dp \times dq &= |dp| |dq| \sin \theta \\ \Rightarrow s &= \frac{|dq|}{|dp|} & \Rightarrow \cos \theta &= \frac{dp \cdot dq}{|dp| |dq|}, \sin \theta = \frac{dp \times dq}{|dp| |dq|}\end{aligned}$$

$$\begin{aligned}\Rightarrow A &= \frac{|dq|}{|dp|} \begin{bmatrix} \frac{dp \cdot dq}{|dp| |dq|} & -\frac{dp \times dq}{|dp| |dq|} \\ \frac{dp \times dq}{|dp| |dq|} & \frac{dp \cdot dq}{|dp| |dq|} \end{bmatrix} \\&= \frac{1}{|dp|^2} \begin{bmatrix} dp \cdot dq & -dp \times dq \\ dp \times dq & dp \cdot dq \end{bmatrix}\end{aligned}$$

$$num = (dp \times dq, dp \cdot dq)$$

$$\Rightarrow A = \frac{1}{|dp|^2} \begin{bmatrix} num_y & -num_x \\ num_x & num_y \end{bmatrix}$$

$$\Rightarrow T(r) = q_0 + A(r - p_0)$$

$$= q_0 + \frac{1}{|dp|^2} \begin{bmatrix} num_y & -num_x \\ num_x & num_y \end{bmatrix} \begin{bmatrix} (r - p_0)_x \\ (r - p_0)_y \end{bmatrix}$$

$$= q_0 + \frac{1}{|dp|^2} \begin{bmatrix} num_y(r - p_0)_x - num_x(r - p_0)_y \\ num_x(r - p_0)_x + num_y(r - p_0)_y \end{bmatrix}$$

$$= q_0 + \frac{1}{|dp|^2} \begin{bmatrix} (r - p_0) \times num \\ (r - p_0) \cdot num \end{bmatrix}$$

### linearTransformation.h

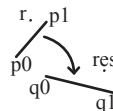
#### Description:

对点 r 应用线性变换（平移、旋转和缩放），该线性变换使得线段 p0-p1 映射到线段 q0-q1。

"Point.h" a5cc19, 6 lines

```
using P = Point<double>;
```

```
P linearTransformation(const P& p0, const P& p1,
    const P& q0, const P& q1, const P& r) {
    P dp = p1-p0, dq = q1-q0, num(dp.cross(dq), dp.dot(dq));
    return q0 + P((r-p0).cross(num), (r-p0).dot(num))/dp.dist2();
}
```



## 7.2 Polygons

### 单调链求二维凸包

先扫一遍 pts，构造下壳，删掉末尾重复点，reverse(pts); 再扫一遍 pts，构造上壳。

ConvexHull.h

#### Description:

返回凸包顶点组成的 vector，顺序为逆时针。位于凸包边上、夹在另外两个顶点之间的共线点，不被视为凸包的一部分。

Time:  $\mathcal{O}(n \log n)$

"Point.h" 0c88c1, 13 lines

```
using P = Point<ll>;
vector<P> convexHull(vector<P> pts) {
    if (sz(pts) <= 1) return pts;
    sort(all(pts));
    vector<P> h(sz(pts)+1);
    int s = 0, t = 0;
    for (int it = 2; it--; s = --t, reverse(all(pts)))
        for (P p : pts) {
            while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t--;
            h[t++] = p;
        }
    return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}
```



## Strings (8)

### KMP.h

**Description:** pi[x] 计算 s 的最长前缀长度，且该前缀以 s[x] 结尾，且不等于 s[0...x] 本身 (abacaba -> 0010123)。可以用来找到一个字符串的所有出现位置。

Time:  $\mathcal{O}(n)$

d4375c, 16 lines

```
vi pi(const string& s) {
    vi p(sz(s));
    rep(i, 1, sz(s)) {
        int g = p[i-1];
        while (g && s[i] != s[g]) g = p[g-1];
        p[i] = g + (s[i] == s[g]);
    }
    return p;
}
```

```
vi match(const string& s, const string& pat) {
    vi p = pi(pat + '\0' + s), res;
    rep(i, sz(p)-sz(s), sz(p))
        if (p[i] == sz(pat)) res.push_back(i - 2 * sz(pat));
    return res;
}
```

### Zfunc.h

**Description:** 维护 [l, r] 表示当前已知最靠右的匹配区间 s[l..r-1] == s[0..r-1-l]。z[i] 计算 s[i] 与 s 的最长公共前缀，但 z[0] = 0 除外。(abacaba -> 0010301)

Time:  $\mathcal{O}(n)$

ee09e2, 12 lines

```
vi Z(const string& S) {
    vi z(sz(S));
    int l = -1, r = -1;
    rep(i, 1, sz(S)) {
        z[i] = i >= r ? 0 : min(r - i, z[i - l]);
        while (i + z[i] < sz(S) && S[i + z[i]] == S[z[i]])
            z[i]++;
        if (i + z[i] > r)
            l = i, r = i + z[i];
    }
    return z;
}
```

## Manacher.h

**Description:** 对于字符串中的每个位置  $i$ ,  $p[0][i]$  = 中心在  $i-1$  与  $i$  之间的最长偶回文半长,  $p[1][i]$  = 中心在  $i$  的最长奇回文半长 (向下取整)(不包括  $i$ )

**Time:**  $\mathcal{O}(N)$  09d7b8, 13 lines

```
array<vi, 2> manacher(const string &s) {
    int n = sz(s);
    array<vi, 2> p = {vi(n + 1, 0), vi(n, 0)};
    rep(id, 0, 2) for (int i = 0, l = 0, r = 0; i < n; i++) {
        int t = r - i + !id;
        if (i < r) p[id][i] = min(t, p[id][l + t]);
        int L = i - p[id][i], R = i + p[id][i] - !id;
        while (L - 1 >= 0 && R + 1 <= n - 1 && s[L - 1] == s[R + 1])
            p[id][i]++, L--, R++;
        if (R > r) l = L, r = R;
    }
    return p;
}
```

## MinRotation.h

**Description:** 找到字符串中字典序最小的循环位移

**Usage:** rotate(v.begin(), v.begin()+minRotation(v), v.end());

**Time:**  $\mathcal{O}(N)$  7c0f7c, 8 lines

```
int minRotation(string s) {
    int a=0, n=sz(s); s += s;
    rep(b, 0, n) rep(k, 0, n) {
        if (a+k == b || s[a+k] < s[b+k]) { b += max(0, k-1); break; }
        if (s[a+k] > s[b+k]) { a = b; break; }
    }
    return a;
}
```

# Various (9)

## 9.1 Misc. algorithms

### TernarySearch.h

**Description:** 在区间  $[a, b]$  中寻找使得  $f(i)$  最大化的最小  $i$ , 假设  $f(a) < \dots < f(i) \geq \dots \geq f(b)$ . 若要反转哪一侧允许非严格不等式, 请将 (A) 处的  $<$  改为  $\leq$ , 并反转 (B) 处的循环. 若要最小化  $f$ , 也在 (B) 处将其改为  $>$ .

**Usage:** int ind = ternSearch(0, n-1, [&](int i){return a[i]});

**Time:**  $\mathcal{O}(\log(b-a))$  9155b4, 11 lines

```
template<class F>
int ternSearch(int a, int b, F f) {
    assert(a <= b);
    while (b - a >= 5) {
        int mid = (a + b) / 2;
        if (f(mid) < f(mid+1)) a = mid; // (A)
        else b = mid+1;
    }
    rep(i, a+1, b+1) if (f(a) < f(i)) a = i; // (B)
    return a;
}
```

### LIS.h

**Description:** 返回最长上升 (非降) 子序列的下标。

**Time:**  $\mathcal{O}(N \log N)$  277a90, 17 lines

```
template<class I> vi lis(const vector<I> &S) {
    if (S.empty()) return {};
    vi prev(sz(S));
    using p = pair<I, int>;
    vector<p> res;
    rep(i, 0, sz(S)) {
        // 把lower_bound中的 0 -> i 以求最长非降子序列
        auto it = lower_bound(all(res), p{S[i], 0});
        if (it == res.end()) res.emplace_back(), it = res.end()-1;
```

## Manacher MinRotation TernarySearch LIS

```
*it = {S[i], i};
prev[i] = it == res.begin() ? 0 : (it - 1)->second;
}
int L = sz(res), cur = res.back().second;
vi ans(L);
while (L-->) ans[L] = cur, cur = prev[cur];
return ans;
}
```

## 9.2 Debugging tricks

- `signal(SIGSEGV, [](int) { _Exit(0); });`; 把段错误转换为 WA。类似地可捕获 `SIGABRT`(断言失败) 和 `SIGFPE` (除以零)。`_GLIBCXX_DEBUG` 失败产生 `SIGABRT` (或在 gcc 5.4.0 上产生 `SIGSEGV`)。
- `feenableexcept(29);` 出现 NaNs (1), 0-divs (4), infinities (8) and denormals (16) 时终止程序。